JSONHTL - JSON Hypertext Language

Version 0.1 — Draft

JSONHTL is a minimal document format encoded as JSON. It is designed to be trivially parseable by LLMs and straightforwardly convertible to HTML or other presentation formats by simple renderers.

JSONHTL defines only primitive building blocks. Higher-level conventions (document metadata, node hierarchies, runnable documents, etc.) are defined in separate convention documents linked from the root node of a given system.

Document Structure

A JSONHTL document is a JSON object. The only required key is `content`.

```
{
  "title": "Example Document",
  "content": [...]
}
```

`title` is optional but conventional. Any other top-level keys are permitted; their meaning is defined by convention documents, not this spec. Renderers and consumers should ignore keys they do not recognise.

Content

`content` is either a **string** (shorthand for a single paragraph of plain text) or a **list of block elements**. When it is a list, each block is a JSON object with a single identifying key.

A string value:

```
{"title": "Quick note", "content": "Remember to update the config."}
```

is equivalent to:

```
{"title": "Quick note", "content": [{"para": ["Remember to update the config."]}]}
```

Block Elements

para A paragraph. Contains a list of **inline elements**.

```
{"para": ["This is plain text with a ", {"link": {"href": "other-node", "text": "link"}}, "
```

heading A section heading. Contains `level` (integer, 1–6) and `text` (string).

```
{"heading": {"level": 1, "text": "Introduction"}}
```

codeblock A block of code or preformatted text. Contains `lang` (string, may be empty) and `body` (string).

```
{"codeblock": {"lang": "python", "body": "print('hello')"}}
```

Additional keys on a codeblock (e.g. `name`, `exec`) are not defined by this spec but may be defined by convention documents.

Inline Elements

Inline elements appear inside `para` lists. An inline element is either a **string** (plain text) or an **object** with a single identifying key.

Plain text A bare JSON string.

```
"This is just text."
```

link A hypertext reference. Contains `href` (string) and `text` (string).

```
{"link": {"href": "getting-started", "text": "Getting Started"}}
```

href resolution:

- Starts with `http://` or `https://` — external web link.
- Anything else — a key in the local data store. The storage layer defines how keys map to documents; JSONHTL does not impose hierarchy or path semantics on keys.

Note: the behaviour when following an external link (e.g. whether the target is another JSONHTL store, a web page, or something else) is not defined by this version of the spec and is reserved for future work.

code Inline code. Contains a string.

```
{"code": "gdata_server.py"}
```

General Rules

1. **Lists or scalars.** Where this spec defines a value as a list, a bare scalar (string, integer) is also acceptable as shorthand for a single-element list. Parsers should normalise to list form internally. For example, `{"para": "just text"}` is equivalent to `{"para": ["just text"]}`.
2. **Unknown keys are ignored.** Blocks, inline elements, and top-level keys that a consumer does not recognise should be silently skipped. This allows convention documents to extend the format without breaking basic renderers.
3. **No presentation markup.** JSONHTL does not define emphasis, bold, underline, font size, or colour. Content is semantic. Presentation is the renderer's concern.

4. **Convention documents over spec changes.** New element types, metadata schemas, and structural conventions should be defined in convention documents stored within the system and linked from the root node — not by extending this spec.

Root Node Convention

When JSONHTL documents are stored in a key-value system, the root node (typically the empty-string key `""`) serves as a bootstrap. It should:

- Use only base JSONHTL elements (as defined in this spec) so any consumer can read it without prior knowledge.
- Briefly describe the link/navigation model.
- Link to convention documents that define any extended keys, metadata schemas, or organisational structures used in this particular system.

An LLM or tool encountering the system for the first time reads the root node, follows only the links it needs, and stops. This keeps context window usage minimal.

Example Document

```
{
  "title": "Project Overview",
  "content": [
    {"heading": {"level": 1, "text": "gdata-server"}},
    {"para": [
      "A FastAPI-based HTTP API for GDBM databases. See ",
      {"link": {"href": "gdata-server/api", "text": "API documentation"}},
      " for endpoint details."
    ]},
    {"para": [
      "Configuration is handled via ",
      {"code": ".gdata_server.yaml"},
      " or environment variables."
    ]},
    {"codeblock": {"lang": "bash", "body": "uvicorn gdata_server:app --host 127.0.0.1 --port
  ]
}
```

Example Root Node

```
{
  "title": "Notes System",
  "content": [
    {"heading": {"level": 1, "text": "Welcome"}},
    {"para": [
```

```
      "This is a JSONHTL document store. Each node is a JSON document keyed by a plain strin
      {"code": "link"},
      " element with the key as the ",
      {"code": "href"},
      " value."
    ]},
    {"heading": {"level": 2, "text": "Conventions"}},
    {"para": [
      "This system uses extended conventions documented here:"
    ]},
    {"para": [
      {"link": {"href": "conventions/metadata", "text": "Metadata keys"}},
      " - describes ",
      {"code": "created"},
      ", ",
      {"code": "type"},
      ", ",
      {"code": "tags"},
      " and other node-level keys."
    ]},
    {"para": [
      {"link": {"href": "conventions/hierarchy", "text": "Hierarchy conventions"}},
      " - how nodes are organised and navigated."
    ]},
    {"para": [
      {"link": {"href": "conventions/runnable", "text": "Runnable documents"}},
      " - extended codeblock keys for executable content."
    ]}
  ]
}
```